

All-in-One Image Restoration Model

Arpit, Ankit, Yash, Priyanshi
ECE Department / Project

November 30, 2025

Abstract

This report documents an integrated image restoration architecture that combines a detail-enhanced convolutional backbone (DEConv / DEA-Net) with a prompt-based adapter block (Prompt Block). We present the architectural blocks, formal equations, shape conventions, intuition, and implementation notes. A PyTorch-style pseudocode appendix is included to ease implementation.

Contents

1	Overview	2
2	Detail-Enhanced Convolution (DEConv)	2
2.1	Motivation and Intuition	2
2.2	Mathematical definition	3
2.3	Difference Convolutions (brief)	3
3	Content-Guided Attention (CGA)	3
3.1	Problem	3
3.2	Procedure	3
4	CGA-based Mixup Fusion	4
4.1	Motivation	4
5	Detail-Enhanced Blocks (DEB / DEAB)	4
6	Prompt Block (PGM + PIM)	4
6.1	Objective	4
6.2	Notation and shapes	4
6.3	Prompt Generation Module (PGM)	4
6.4	Prompt Interaction Module (PIM)	5
7	Network-level dataflow and shape bookkeeping	5
8	Training objective	5
9	Implementation notes and recommendations	6
10	Appendix A: PyTorch-style pseudocode	6
11	Appendix B: Example hyperparameters	7
12	Concluding remarks	7

1 Overview

The overall model is a three-level encoder-decoder (UNet-like) architecture with the following parts:

- **Encoder:** multi-level feature extraction using Detail-Enhanced Blocks (DEB).
- **Feature transform:** deeper blocks (DEAB) to progressively transform features at the bottleneck level.
- **Decoder:** feature upsampling, CGA-based fusion with encoder features and final reconstruction conv.
- **Prompt Blocks:** inserted at decoder levels to provide degradation-aware guidance via learnable prompts and a lightweight transformer interaction.

A high-level schematic is shown in Figure 1.

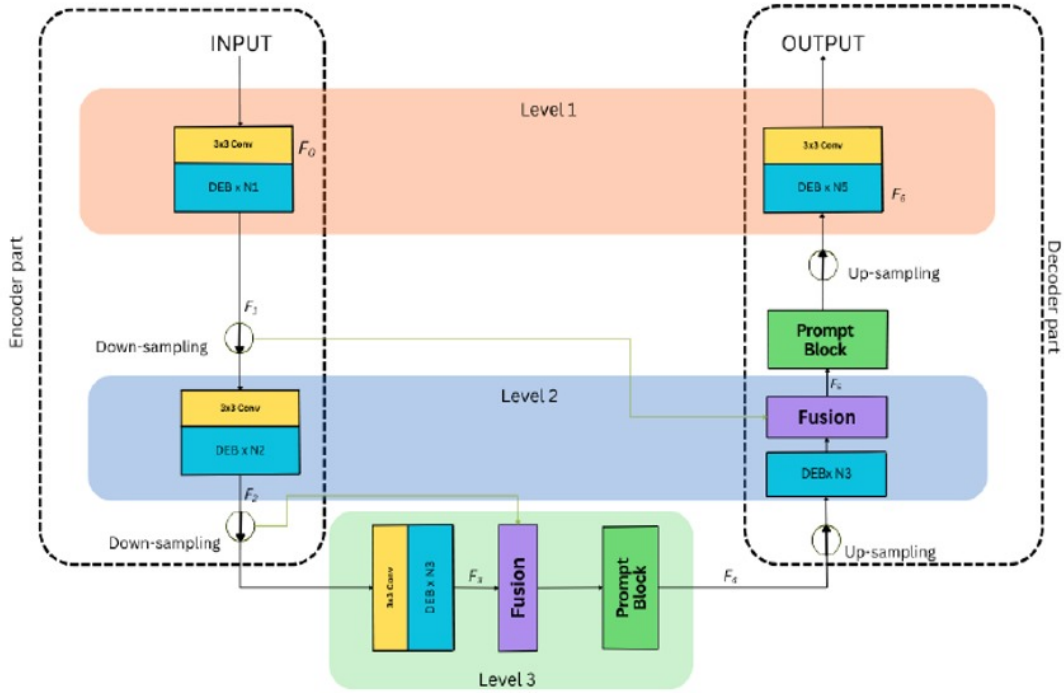


Figure 1: Overall architecture (encoder, DEConv blocks, CGA, prompt block and decoder).

2 Detail-Enhanced Convolution (DEConv)

2.1 Motivation and Intuition

Standard convolution learns intensity filters but does not explicitly enforce gradient/edge priors. For image restoration (e.g., dehazing, denoising, deblurring) high frequency (edge/texture) cues are crucial. DEConv integrates multiple difference convolutions in parallel with a vanilla convolution to explicitly capture both intensity and gradient-level features.

2.2 Mathematical definition

Let the input feature map be F_{in} . DEConv consists of $M = 5$ parallel convolutions:

$$\text{DEConv}(F_{\text{in}}) = \sum_{i=1}^5 F_{\text{in}} \star K_i,$$

where K_1 is the vanilla conv kernel (VC), and $K_2 \dots K_5$ denote kernels for CDC, ADC, HDC, VDC respectively, and $\star$ is convolution. For simplicity we omit biases.

Because the outputs of the branches are summed, one can re-parameterize:

$$K_{\text{cvt}} = \sum_{i=1}^5 K_i, \quad \text{DEConv}(F_{\text{in}}) = F_{\text{in}} \star K_{\text{cvt}}.$$

During training keep the five kernels separate (so gradients flow into each prior); during inference collapse them into K_{cvt} to get standard convolution cost.

2.3 Difference Convolutions (brief)

- **Central Difference Convolution (CDC):** emphasizes center-surround differences by combining standard conv with a weighted sum of local differences.
- **Angular Difference Convolution (ADC):** encodes angular/pairwise pixel differences.
- **Horizontal / Vertical Difference Convolution (HDC / VDC):** engineered to resemble Sobel/Prewitt kernels which specialize in horizontal/vertical gradient extraction.

Each DC can be realized by suitable re-arrangement of learned kernel weights to compute pixel differences efficiently.

3 Content-Guided Attention (CGA)

3.1 Problem

Standard FAM (channel + spatial attention) is limited: channel attention lacks local spatial guidance per channel; spatial attention is shared across channels. We need channel-specific spatial importance maps (SIMs).

3.2 Procedure

Given $X \in \mathbb{R}^{C \times H \times W}$:

$$W_c = \text{Conv}_{1 \times 1}(\text{ReLU}(\text{Conv}_{1 \times 1}(\text{GAP}(X)))) \in \mathbb{R}^{C \times 1 \times 1}, \quad (1)$$

$$W_s = \text{Conv}_{7 \times 7}([\text{GAP}_{\text{spatial}}(X); \text{GMP}_{\text{spatial}}(X)]) \in \mathbb{R}^{1 \times H \times W}, \quad (2)$$

broadcast/add them:

$$W_{\text{coa}} = W_c + W_s \quad (\text{broadcasted to } C \times H \times W).$$

Refine each channel by concatenating X with W_{coa} , applying channel-shuffle and grouped convolution + sigmoid:

$$W = \sigma(\text{GConv}_{7 \times 7}(\text{CS}([X; W_{\text{coa}}]))), \quad W \in \mathbb{R}^{C \times H \times W}.$$

Finally the output is $X' = X \odot W$ (element-wise).

4 CGA-based Mixup Fusion

4.1 Motivation

Direct addition / concatenation of shallow + deep features can suffer from receptive field mismatch. We compute adaptive spatial weights W from both shallow (F_{low}) and deep (F_{high}) features and perform a mixup:

$$F_{\text{fuse}} = \text{Conv}_{1 \times 1}(F_{\text{low}} \odot W + F_{\text{high}} \odot (1 - W) + F_{\text{low}} + F_{\text{high}}).$$

This both modulates contributions channel/space-wise and preserves skip connections to ease gradient flow.

5 Detail-Enhanced Blocks (DEB / DEAB)

DEB (Detail-Enhanced Block) Uses DEConv + CGA (optional) in a residual structure. Output refines both intensity and gradient cues.

DEAB (Detail-Enhanced Attention Block) Augments DEB with the *CGA* and additional gating mechanisms in the bottleneck level (level 3) for more expressive transforms.

Block diagram (conceptual):

$$\text{DEB}(X) = X + \text{ReLU}(\text{DEConv}(X)) \odot \text{CGA}(X).$$

6 Prompt Block (PGM + PIM)

6.1 Objective

Provide degradation-specific guidance to decoder features via learnable prompt components that are dynamically weighted per input.

6.2 Notation and shapes

Let:

$$F_l \in \mathbb{R}^{B \times C \times H \times W}, \quad P_c \in \mathbb{R}^{N \times C \times \tilde{H} \times \tilde{W}},$$

(we will upsample P_c to (H, W) if needed). The Prompt Block computes:

$$\hat{F}_l = \text{PIM}(\text{PGM}(P_c, F_l), F_l).$$

6.3 Prompt Generation Module (PGM)

1. Global average pooling across spatial dims:

$$v = \text{GAP}(F_l) \in \mathbb{R}^{B \times C}.$$

2. Channel downscaling via 1×1 conv (or FC) and softmax to obtain prompt weights:

$$w = \text{Softmax}(\text{Conv}_{1 \times 1}(v)) \in \mathbb{R}^{B \times N}.$$

3. Weighted combination of prompt components (upsampled to target spatial size) and a 3×3 conv:

$$P = \text{Conv}_{3 \times 3} \left(\sum_{i=1}^N w_i \text{Upsample}(P_{c,i}) \right) \in \mathbb{R}^{B \times C \times H \times W}.$$

6.4 Prompt Interaction Module (PIM)

1. Concatenate prompt and feature along channel dimension:

$$U = [F_l; P] \in \mathbb{R}^{B \times 2C \times H \times W}.$$

2. Normalize and project to compute Q, K, V via 1×1 pointwise convs followed by 3×3 depthwise convs:

$$Q = \text{DWConv}_{3 \times 3}(\text{Conv}_{1 \times 1}(\text{LN}(U))), \quad K = \text{DWConv}_{3 \times 3}(\text{Conv}_{1 \times 1}(\text{LN}(U))), \quad V = \text{DWConv}_{3 \times 3}(\text{Conv}_{1 \times 1}(\text{LN}(U))).$$

3. MDTA aggregation (attention across channels, linear in spatial size):

$$A = \text{Softmax}\left(\frac{K \cdot Q^\top}{\alpha}\right), \quad Y = W_p V \cdot A + U,$$

where W_p is a pointwise conv and α is a learnable scaling.

4. GDFN transform (gated depthwise conv FFN):

$$Z = W_0^p \left(\varphi(W_1^d W_1^p(\text{LN}(Y))) \odot W_2^d W_2^p(\text{LN}(Y)) \right) + Y,$$

where W^p are pointwise convs, W^d depthwise convs, φ is GELU and $\odot$ element-wise multiplication.

5. Project back to C channels with a final 3×3 conv:

$$\hat{F}_l = \text{Conv}_{3 \times 3}(Z) \in \mathbb{R}^{B \times C \times H \times W}.$$

7 Network-level dataflow and shape bookkeeping

We adopt the following level shapes (channels scale by factor 2 each downsampling):

$$\text{Level 1: } F_1 \in \mathbb{R}^{B \times C \times H \times W},$$

$$\text{Level 2: } F_2 \in \mathbb{R}^{B \times 2C \times H/2 \times W/2},$$

$$\text{Level 3 (bottleneck): } F_3 \in \mathbb{R}^{B \times 4C \times H/4 \times W/4}.$$

Downsampling halves spatial dims and doubles channels; upsampling mirrors this.

At each decoder level we optionally place a Prompt Block that takes the decoder feature F_l and returns $\hat{F}_l$ of same shape, then fuses $\hat{F}_l$ with the corresponding encoder feature using CGA-based mixup and continues decoding.

8 Training objective

The base training loss used in DEA-Net is L_1 loss:

$$\mathcal{L}_{\ell_1} = \frac{1}{BHW} \sum_{b,h,w} |J_{\text{pred}}(b, h, w) - J_{\text{gt}}(b, h, w)|.$$

You may optionally augment with:

- Perceptual loss (VGG feature space) to preserve perceptual quality.
- Edge / gradient loss to further enforce sharp boundaries.
- Frequency loss (FFT) to match spectral content.

9 Implementation notes and recommendations

- **DEConv training/inference:** keep five branches separate during training; before inference sum the kernels to form a single conv.
- **Prompt Components Initialization:** initialize P_c small (e.g., Gaussian with std 0.01). N (number of components) depends on expected degradation diversity (4–16 typical).
- **PGM bottleneck:** use small hidden size before Softmax (e.g., reduce channels to 64 then to N).
- **Normalization:** use LayerNorm inside Transformer blocks and possibly BatchNorm in convs.
- **Upsampling prompts:** bilinear upsample P_c to the current feature spatial size before combining.
- **Computational cost:** MDTA operates mainly across channels (linear w.r.t. spatial size) which helps efficiency for larger images.

10 Appendix A: PyTorch-style pseudocode

This is conceptual pseudocode (not fully runnable). Use as blueprint.

```
class DEConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.vc = nn.Conv2d(in_ch, out_ch, 3, padding=1)
        self.cdc = ConvForCDC(in_ch, out_ch) # implement as rearranged kernel
        self.adc = ConvForADC(...)
        self.hdc = ConvForHDC(...)
        self.vdc = ConvForVDC(...)
    def forward(self, x):
        out = self.vc(x) + self.cdc(x) + self.adc(x) + self.hdc(x) + self.vdc(x)
        return out
    # Optionally provide a method to reparam for inference:
    def reparam(self):
        ksum = sum(branch.weight for branch in [self.vc, self.cdc, ...])
        self.collapse_conv = nn.Conv2d(...); self.collapse_conv.weight = ksum

class PGM(nn.Module):
    def __init__(self, C, N, prompt_shape):
        super().__init__()
        self.prompts = nn.Parameter(torch.randn(N, C, *prompt_shape))
        self.fc = nn.Conv1d(C, N, 1) # or small MLP
        self.post = nn.Conv2d(C, C, 3, padding=1)
    def forward(self, F1):
        v = F1.mean(dim=[2,3]) # B x C
        w = torch.softmax(self.fc(v.unsqueeze(-1)).squeeze(-1), dim=-1) # B x N
        P_i = upsample_prompts(self.prompts, F1.shape[2:])
        P_weighted = (w.unsqueeze(2).unsqueeze(3).unsqueeze(4) * P_i).sum(dim=1)
        P = self.post(P_weighted)
        return P
```

```

class PromptBlock(nn.Module):
    def __init__(...):
        self.pgm = PGM(...)
        self.pim_transformer = SmallMDTAGDFN(...)
        self.out_conv = nn.Conv2d(2*C, C, 3, padding=1)
    def forward(self, F1):
        P = self.pgm(F1)
        U = torch.cat([F1, P], dim=1)
        U2 = self.pim_transformer(U)
        return self.out_conv(U2)

# Build encoder/decoder using DEB/DEAB and insert PromptBlock in decoder levels.

```

11 Appendix B: Example hyperparameters

- Base channels $C = 32$.
- Number of prompt components $N = 8$.
- Optimizer: AdamW, $\text{lr} = 2 \times 10^{-4}$, weight decay $= 1 \times 10^{-6}$.
- Loss: L_1 base. Optionally add $\lambda_{\text{edge}} = 0.05$, $\lambda_{\text{freq}} = 0.1$.
- Training crops: 256×256 .

12 Concluding remarks

The combination of DEConv (to capture intensity+gradient priors), CGA (to produce channel-wise SIMs), CGA-based fusion (to mitigate receptive field mismatch), and Prompt Blocks (to make the decoder degradation-aware) yields a powerful, modular architecture for low-level image restoration tasks. The prompt module is plug-in and architecture-agnostic — you can add it to other restoration backbones to provide input-conditioned guidance.

References: include references for CDC, ADC, LBC, MDTA, etc., e.g., Xu et al. (LBC), Yu et al. (CDC), and the PromptIR paper that introduced MDTA/GDFN.